

Reviewer Pack — Minimal Barycentric Coordinates and Circuit Entanglement Com...

Entry 3ec0007ae4ae · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 13:07:23 UTC

Minimal Barycentric Coordinates and Circuit Entanglement Complexity

Entry ID: 3ec0007ae4ae **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 13:07:23 UTC

1. Conjecture

Field A (mathematical branch): Computational Geometry (Barycentric Coordinates) **Field B** (complexity object): Boolean Circuits (Circuit Complexity)

Statement:

For every n -input Boolean circuit C , the minimum number of barycentric coordinates required to represent its input space is linearly correlated with its entanglement complexity, such that $|BC(C)| = \Theta(EC(C))$, where $BC(C)$ denotes the minimal set of barycentric coordinates for C and $EC(C)$ is the circuit entanglement complexity.

Rationale (proposer's reasoning):

Barycentric coordinates provide a means to embed points in a higher-dimensional space into a lower dimension while preserving certain properties, which may relate to the structure of circuits. This could expose inherent geometric structures within Boolean functions that influence their computational complexity.

Taxonomy category: Computational_Geometry_to_Boolean_Circuits (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3784ef00b0ffe603

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 95% of the tested circuits, the ratio of the minimum number of barycentric coordinates to the entanglement complexity is within $\pm 10\%$ of 1. The conjecture is falsified if this ratio exceeds $\pm 20\%$ for any circuit.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "barycentric coordinates" AND "Boolean circuits" AND entanglement complexity - minimal barycentric coordinates IN computational geometry AND correlation WITH Boolean circuit complexity - linear relationship between minimal set of barycentric coordinates AND circuit entanglement

Top relevant hits considered: - [<http://arxiv.org/abs/cs/0403008v3>] Polynomial-time computing over quadratic maps I: sampling in real algebraic sets - [<http://arxiv.org/abs/1712.04281v2>] Computational Complexity of the Interleaving Distance - [<http://arxiv.org/abs/2111.12700v2>] Universality in long-distance geometry and quantum complexity - [<http://arxiv.org/abs/1911.09867v2>] Full Characterization of Minimal Linear Codes as Cutting Blocking Sets - [<http://arxiv.org/abs/1109.5531v1>] On the relationship between classes P and NP - [<http://arxiv.org/abs/2505.23318v2>] Minimal displacement set for CAT(0) cubical complexes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_circuit(n):
        circuit = []
        for _ in range(2**n - 1):
            gate_type = random.choice(['AND', 'OR'])
            inputs = sorted(random.sample(range(n), 2))
            circuit.append((gate_type, inputs))
        return circuit

    def compute_entanglement_complexity(circuit):
        n = len(circuit) + 1
        complexity = 0
        for gate in circuit:
            if gate[0] == 'AND':
                complexity += 1
            elif gate[0] == 'OR':
                complexity += 1
        return complexity

    def compute_barycentric_coordinates(circuit):
```

```

n = len(circuit) + 1
coordinates = set()
for i in range(2**n):
    point = [int(x) for x in format(i, f'0{n}b')]
    valid = True
    for gate in circuit:
        if gate[0] == 'AND':
            if point[gate[1][0]] + point[gate[1][1]] != 2:
                valid = False
                break
        elif gate[0] == 'OR':
            if point[gate[1][0]] + point[gate[1][1]] != 1:
                valid = False
                break
    if valid:
        coordinates.add(tuple(point))
return len(coordinates)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    circuit = generate_random_circuit(n)
    entanglement_complexity = compute_entanglement_complexity(circuit)
    barycentric_coordinates = compute_barycentric_coordinates(circuit)
    results.append((n, entanglement_complexity, barycentric_coordinates))

total_n = sum(1 for n, _, _ in results)
mean_ratio = sum(b / e for n, e, b in results) / total_n
std_dev = math.sqrt(sum((b / e - mean_ratio)**2 for n, e, b in results) / total_n)

support_fraction = sum(abs(b / e - 1) <= 0.1 for n, e, b in results) / total_n

return {
    "metric_name": "Ratio of Barycentric Coordinates to Entanglement Complexity",
    "metric_value": mean_ratio,
    "instances_tested": total_n,
    "n_max": max(n for n, _, _ in results),
    "conjecture_holds": support_fraction >= 0.95,
    "counterexample": "" if support_fraction >= 0.95 else "Ratio exceeds ±20%"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [3, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_ratio)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if abs(result["metric_value"] - 1) <= 0.1) / len(results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
    elif any(abs(result["metric_value"] - 1) > 0.2 for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if abs(result["metric_value"] - 1) > 0.2)

```

```

print(f"RESULT: FALSIFIED counterexample='Ratio exceeds  $\pm 20\%$ ' first_failing_seed={first_fa
else:
print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete its execution and provide results. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13291
2	propose	ollama_remote	glm4:latest	0	0	12606
3	preregistration	ollama_remote	glm4:latest	0	0	9370
4	novelty	ollama_remote	glm4:latest	0	0	13561
5	novelty	ollama_remote	glm4:latest	0	0	9961
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14950
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9948
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12292
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11874
10	judge	ollama_remote	glm4:latest	0	0	36380

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 144233 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/3ec0007ae4ae.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/3ec0007ae4ae.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3ec0007ae4ae.tar.gz` (if generated)